

نظام قاعدة بيانات مستشفى آمن مع التحكم بالصلاحيات (RBAC)

Secure Online Hospital Database Management System — PostgreSQL · Node.js/Express

التاريخ: 29-06-2026

عدد الأسئلة: 42

المادة: قواعد البيانات / Database Systems

الفريق: [تُدرج أسماء الفريق الخمسة هنا]

كيف تستخدم هذا الدفتر

- افهم لا تحفظ: كل إجابة تشرح «لماذا» لا «ماذا» فقط — استوعب المنطق لتجيب على أي صياغة.
- الوسم الملون: سهل أساسيات متوقعة · متوسط فهم تطبيقي · صعب أسئلة تمييزية.
- درّب نفسك: اقرأ السؤال، أجب بصوتك، ثم قارن بالإجابة النموذجية.
- اربط بالكود: كل إجابة تشير لجزء فعلي في مشروعك — افتح الملف وتأكد أنك تراه.

المحاور (6)

1. تصميم قاعدة البيانات — Data Model & ERD سؤال
2. التطبيق والاستعلامات — Normalization & SQL سؤال
3. التحكم بالصلاحيات — RBAC & Row-Level Security سؤال
4. الأمان والتدقيق — Security & Auditing سؤال
5. الواجهة الخلفية — Backend & REST API سؤال
6. أسئلة عامة ومناقشة — General Viva سؤال

لماذا فصلت جدول users (المصادقة) عن جدولَي staff و patients بدلاً من وضع username و password_hash مباشرةً داخلهما؟ (Authentication vs. Identity separation)

الإجابة النموذجية

الفصل مقصود لثلاثة أسباب:

- 1) فصل المسؤوليات (Separation of Concerns): جدول users مسؤول عن المصادقة فقط — يحمل username, email, password_hash (bcrypt/argon2), is_active, failed_login_attempts, last_login_at. أما staff و patients فيحملان البيانات الديموغرافية والمهنية. هذا يجعل كل جدول له غرض واحد واضح.
 - 2) ليس كل شخص في النظام يملك حساب دخول: المريض قد يُسجل في patients (يأتي للعيادة) دون أن يملك حساب بوابة (portal account)، والموظف قد يكون في staff دون حساب. لو دمجتنا المصادقة في patients لأجبرنا كل مريض على امتلاك username/password، وهذا غير واقعي.
 - 3) الأمان (Security): حصر الحقول الحساسة (password_hash, failed_login_attempts) في جدول واحد يسهل حمايته بـ RLS و audit، ويسهل استبعاد password_hash من سجل التدقيق، ويتيح آلية القفل ضد التخمين (brute-force lockout) في مكان واحد.
- الربط يتم عبر users.patient_id و users.staff_id (كلاهما nullable)، مع قيد CHECK (staff_id IS NOT NULL OR patient_id IS NOT NULL). وهذا الفصل هو جوهر آلية RBAC: من users.id نستقّ doctor_id أو patient_id داخل سياسات RLS لتطبيق قاعدة (الطبيب يرى مرضاه فقط / المريض يرى سجله فقط).

اشرح أنواع العلاقات الموجودة في تصميمك مع مثال على كل منها: واحد-لواحد (1:1)، واحد-لكثير (N:1)، وكثير-لكثير (N:M).

الإجابة النموذجية

التصميم يحوي الأنواع الثلاثة:

- 1:1 (One-to-One): doctors مع staff، و nurses مع staff. كل طبيب هو موظف واحد بالضبط، والعكس صحيح. تُفرض تقنياً عبر `staff_id BIGINT NOT NULL UNIQUE` في `doctors/nurses` — قيد `UNIQUE` هو ما يحوّل العلاقة من 1:N إلى 1:1 (يمنع أن يرتبط أكثر من سجل `doctors` بنفس الموظف). الغرض: فصل البيانات التخصصية (`specialty`, `license_number`) عن الجدول العام `staff` بدل جدول منتفخ بأعمدة `NULL`.
- 1:N (One-to-Many): patient:1 medical_records. N واحد له عدّة `medical_records`. تُمثّل بوضع المفتاح الأجنبي في جهة الكثير: `patients(id) REFERENCES medical_records.patient_id`. أمثلة أخرى: `department` واحد فيه عدّة `staff`; `invoice` واحدة لها عدّة `payments` (دعم السداد الجزئي)؛ `medical_record` واحد له عدّة `diagnoses`.
- N:M (Many-to-Many): مريض له عدّة أطباء وطبيب له عدّة مرضى. لا يمكن تمثيلها بمفتاح أجنبي مباشر، لذا نستخدم جدول وصل (junction table) — `appointments` هنا يجسّد العلاقة (`patient_id + doctor_id` + بيانات الموعد). أمثلة أخرى صريحة على جداول الوصل: `prescription_items` (`roles × permissions`), `role_permissions` (`users × roles`), `user_roles` (`prescriptions × medications`), `lab_orders` (`medical_records × lab_tests`).

ما جدول الوصل (junction / bridge table)؟ ولماذا لا يمكن تمثيل علاقة N:M بدونه؟ خذ prescription_items مثالاً.

الإجابة النموذجية

جدول الوصل هو جدول وسيط يحلّ علاقة كثير-لكثير (N:M) بتقسيمها إلى علاقتي 1:N. السبب أنه في قاعدة بيانات علائقية لا يمكن لعمود واحد أن يحمل قائمة قيم (مبدأ الذرية / 1NF) — فلا يمكن أن نضع في prescriptions عموداً يحمل عدّة medication_id، ولا أن نضع في medications عموداً يحمل عدّة prescription_id. الدواء الواحد يظهر في آلاف الوصفات، والوصفة الواحدة تحوي عدّة أدوية، فالعلاقة N:M.

الحل: prescription_items يحمل (prescription_id, medication_id) كمفتاحين أجنبيين، فيصبح:

prescriptions 1:N prescription_items -

medications 1:N prescription_items -

وحاصل العلاقتين = N:M بين الوصفات والأدوية.

الميزة الإضافية: جدول الوصل ليس مجرد ربط بل يحمل بيانات خاصة بالعلاقة (relationship attributes): dosage, frequency, duration_days, quantity, instructions — وهي بيانات لا تخصّ الدواء وحده ولا الوصفة وحدها بل تخصّ تركيبتهما معاً. وأضفتُ UNIQUE (prescription_id, medication_id) لمنع تكرار نفس الدواء في الوصفة الواحدة. نفس المبدأ ينطبق على role_permissions و user_roles في طبقة RBAC.

اشرح سياسات ON DELETE الثلاث التي استخدمتها (CASCADE / RESTRICT / SET NULL)، ولماذا اخترت كلًا منها لجداول معيّن؟ أعطِ مثالًا يبيّن خطورة الاختيار الخاطئ.

الإجابة النموذجية

ON DELETE تحدد ما يحدث للسجلات التابعة عند حذف السجل الأب، واختارها وفق طبيعة كل علاقة (referential integrity strategy):

(1 ON DELETE CASCADE — حذف السجل الأب يحذف التابع تلقائياً. استخدمتها للبيانات التي لا معنى لها بدون أبيها: diagnoses عند حذف medical_record؛ prescription_items عند حذف prescription؛ invoice_items و payments عند حذف invoice؛ user_roles عند حذف user. بند فاتورة بلا فاتورة = بيانات يتيمّة (orphan) لا معنى لها.

(2 ON DELETE RESTRICT — يمنع حذف الأب طالما له تابع. استخدمتها للمراجع الحرجة التي يؤدي فقدانها لكارثة: medical_records.doctor_id (لا يجوز حذف طبيب وله سجلات طبية فنفقد نسبة السجل)؛ prescription_items.medication_id (لا تُحذف دواء مذكور في وصفة)؛ invoices.patient_id (لا يُحذف مريض له فاتورة مستحقّة). RESTRICT يحمي النزاهة الطبية والمالية.

(3 ON DELETE SET NULL — يجعل المفتاح الأجنبي NULL عند حذف الأب، ويُستخدم للروابط الاختيارية: staff.department_id (موظف بلا قسم وارد)؛ medical_records.appointment_id (السجل يبقى وإن حُذف الموعد)؛ created_by في invoices/appointments. الشرط أن يكون العمود nullable.

مثال على خطورة الاختيار الخاطئ: لو وضعنا ON DELETE CASCADE على medical_records.doctor_id بدل RESTRICT، فإن حذف حساب طبيب استقال سيمحو كل السجلات الطبية لمرضاه — كارثة قانونية وطبية. لذا الحساس يكون RESTRICT (أو soft-delete عبر is_active بدل الحذف الفعلي).

لماذا اخترت BIGSERIAL مفتاحاً أساسياً (surrogate key) بدل INTEGER/SERIAL أو مفتاح طبيعي مثل national_id؟

الإجابة النموذجية

BIGSERIAL هو نوع في PostgreSQL يُنشئ عموداً (64-bit) BIGINT مع تسلسل تلقائي (auto-increment sequence)، أي مفتاح بديل (surrogate key) مولّد آلياً لا معنى له خارج النظام.

لماذا surrogate key لا natural key (مثل national_id):

- المفتاح الطبيعي قد يتغير أو يكون NULL أو غير فريد دائماً، وتغييره يفرض تحديثه في كل الجداول المرتبطة (cascade مؤلم). أما id الاصطناعي فثابت لا يتغير أبداً، ومستقر كهدف للمفاتيح الأجنبية.
- الخصوصية: استخدام national_id مفتاحاً أجنبياً سينشره في كل الجداول ويعرض PII حساساً للتسرب. id رقم محايد آمن للنشر.
- البساطة والأداء: مفتاح عددي واحد أصغر وأسرع في الفهرسة والـ JOIN من مفتاح نصي مركّب.

لماذا BIGSERIAL لا SERIAL (INT 32-bit):

- سقف 2.1~ SERIAL مليار سجل، وهو قابل للنفاذ في سجل طبي/تدقيق ينمو بسرعة (audit_logs خاصة). BIGSERIAL سقفه 9.2~ كوئيتليون عملياً لا ينفد.
- الاتساق: توحيد كل المفاتيح على BIGINT يجنب أخطاء عدم تطابق الأنواع (type mismatch) بين PK و FK في الـ JOIN، ويتجنب تحليلاً مؤلماً لاحقاً من INT إلى BIGINT.

ملاحظة: في PostgreSQL الحديث يُفضّل GENERATED ALWAYS AS IDENTITY على SERIAL معيارياً، لكن BIGSERIAL خيار صحيح وواضح ومتوافق.

إلى أي مستوى تطبيع (Normal Form) وصل تصميمك؟ اشرح كيف حققت 1NF و 2NF و 3NF، ومتى يكون إلغاء التطبيع (denormalization) مقبولاً؟

الإجابة النموذجية

التصميم مطبّع حتى الصورة الطبيعية الثالثة (3NF):

• 1NF (الذرية): كل خلية تحمل قيمة واحدة لا قائمة ولا مجموعات متكرّرة (no repeating groups). لهذا لم أضع عمود (الأدوية) كقائمة في prescriptions، بل فصلت كل دواء في سطر مستقل في prescription_items. ولا يوجد عمود (الهواتف) مكرّر بل قيمة واحدة لكل عمود.

• 2NF (لا تبعية جزئية): في الجداول ذات المفتاح المركّب، كل عمود غير مفتاحي يعتمد على كامل المفتاح لا على جزء منه. لهذا فصلت البنود عن رؤوسها: dosage و frequency تعتمد على (prescription_id, medication_id) معاً فوّضت في prescription_items؛ وبيانات الفاتورة العامة (status, due_date) في invoices لا تتكرر مع كل بند.

• 3NF (لا تبعية انتقالية): لا يعتمد عمود غير مفتاحي على عمود غير مفتاحي آخر. لهذا لم أكرّر اسم الدواء وسعره داخل prescription_items، بل أبقىها في كتالوج medications ويشار إليه بـ medication_id فقط. نفس المبدأ مع lab_tests و lab_orders. هذا يمنع شذوذ التحديث (update anomaly): تغيير سعر الدواء يتم في مكان واحد لا في آلاف البنود.

متى يُقبل إلغاء التطبيع (denormalization) المتحكّم به: عندما يُحسّن الأداء (read performance) بضمن مقبول من التكرار القدار. في تصميمي مثالان:

1 invoice_items.line_total كعمود محسوب STORED ALWAYS AS (quantity * unit_price) — تكرار مشتقّ لكنه آمن لأن قاعدة البيانات تضمن اتساقه تلقائياً (لا يُكتب يدوياً).

2 invoices.total_amount يمكن تخزينه كـ cache للإجمالي تجنّباً لإعادة الجمع عند كل قراءة، شرط تحديثه عبر trigger ليبقى متّسقاً مع البنود.

القاعدة: ابدأ مطبّعاً (3NF)، ولا تُلغ التطبيع إلا بدليل أداء واضح ومع ضمان الاتساق.

صف ال ERD العام لنظامك بالكتل الرئيسية، وكيف عالجت التبعية الدائرية (circular dependency) بين departments و staff عند إنشاء الجداول؟

الإجابة النموذجية

ال ERD ينتظم في خمس كتل وظيفية (22 جدولاً):

- كتلة RBAC: permissions —> role_permissions —> roles و user_roles —> users (علامة <—> تعني جدول وصل (N:M).
 - كتلة الموظفين: departments 1:N staff و staff 1:1 doctors, staff 1:1 nurses.
 - كتلة المصادقة: users 1:1 patients و users 1:1 patients (عبر staff_id/patient_id).
 - الكتلة السريرية: appointments —> doctors :patients 1:N medical_records :patients 1:N medical_records 1:N medical_records 1:N prescriptions 1:N prescription_items N:1 medications :diagnoses lab_orders N:1 lab_tests.
 - الكتلة المالية والتدقيق: invoice_items :invoices 1:N patients 1:N invoices 1:N payments :audit_logs :users 1:N.
- التبعية الدائرية بين departments و staff: المشكلة أن staff.department_id يشير إلى departments(id)، وفي الوقت نفسه departments.head_staff_id (رئيس القسم) يشير إلى staff(id). فأيهما نُشئ أولاً؟ كلٌّ يحتاج الآخر موجوداً لِتُنشئ مفتاحه الأجنبي.

الحل المطبق في schema.sql_01:

- أنشئ departments أولاً بعمود BIGINT head_staff_id لكن بدون قيد FK.
 - أنشئ staff مع FK على department_id (departments صار موجوداً).
 - ثم أضيف FK المتبقي عبر ALTER TABLE departments ADD CONSTRAINT fk_departments_head_staff FOREIGN KEY (head_staff_id) REFERENCES staff(id) ON DELETE SET NULL.
- هكذا نكسر الحلقة بتأجيل أحد القيدين إلى ما بعد إنشاء الجدولين. وملاحظة: head_staff_id يستخدم SET NULL لأن رئاسة القسم رابط اختياري — حذف موظف رئيس لا يجوز أن يحذف القسم، فقط تُفَرِّغ الرئاسة.

② التطبيق والاستعلامات — Normalization & SQL

ما المقصود بالتطبيع (Normalization)، وما الفائدة العملية منه في مشروعك؟ واذكر باختصار ما هي 1NF و 2NF و 3NF.

الإجابة النموذجية

التطبيع (Normalization) هو عملية تنظيم الجداول لتقليل تكرار البيانات (data redundancy) ومنع شذوذ التحديث/الإدراج/الحذف (update/insert/delete anomalies). الفائدة العملية في نظام المستشفى: لو كُزنا بيانات الطبيب أو الدواء في كل سجل، فأني تعديل (مثل تغيير سعر دواء) يتطلب تحديث عشرات الصفوف وقد ينتج تناقض. التطبيع يجعل كل حقيقة مخزنة في مكان واحد فقط (single source of truth).

الدرجات الثلاث باختصار:

- 1NF (First Normal Form): كل خلية ذرية (atomic) لا تحوي قيمة متعددة أو مجموعات متكررة، ولكل صف مفتاح يميزه. أي: لا أعمدة مثل medications_list نضع فيها عدة أدوية مفصولة بفواصل.
 - 2NF: تتحقق 1NF + كل عمود غير مفتاحي يعتمد على كامل المفتاح الأساسي لا على جزء منه (إزالة الاعتماد الجزئي / partial dependency). تظهر أهميتها في الجداول ذات المفتاح المركب.
 - 3NF: تتحقق 2NF + لا يوجد اعتماد انتقالي (transitive dependency)، أي لا يعتمد عمود غير مفتاحي على عمود غير مفتاحي آخر. كل عمود يعتمد على المفتاح، كامل المفتاح، ولا شيء سوى المفتاح.
- في مشروعي قاعدة البيانات مطبوعة حتى 3NF كما هو موضح في design_notes.

أعطني مثالاً من قاعدة بياناتك على تطبيق التطبيع: جدول كان يمكن أن يخالف القواعد ثم كيف طبّعته؟

الإجابة النموذجية

المثال الأوضح هو الوصفة الطبية (prescriptions / prescription_items) والفاتورة (invoices / invoice_items). التصميم غير المطبّع (Unnormalized): جدول prescription واحد فيه أعمدة مثل medication1, dosage1, medication2, dosage2... هذا يخالف 1NF لأنه مجموعات متكررة (repeating groups)، ويهدر مساحة عند الوصفة القصيرة، ويعجز عن استيعاب وصفة بعدد أدوية أكبر من الأعمدة المتاحة، ويصعب الاستعلام (لا يمكن عمل WHERE على دواء معين بسهولة). الحل (Normalization إلى 1NF): فصلت البنود في جدول مستقل prescription_items، كل دواء سطر مستقل بـ dosage و frequency و duration خاصة به، والعلاقة 1:N من prescriptions إلى prescription_items. هذا حوّل علاقة N:M بين medications و prescriptions إلى جدول وصل (junction table) قابل للاستعلام. أمثلة 3NF (إزالة الاعتماد الانتقالي): لم أضع اسم الدواء وسعره داخل prescription_items؛ بل وضعتها في كتالوج medications، ويشير البند إليه عبر medication_id فقط. لو كُزرت اسم الدواء في كل بند لكان اسم الدواء يعتمد على medication_id لا على مفتاح البند = اعتماد انتقالي يخالف 3NF. نفس المبدأ مع lab_tests و lab_orders.

اشرح كيف ظهرت 2NF تحديداً في تصميمك، فجداول كثيرة لديك مفتاحها id بسيط لا مركب، فمتى تكون 2NF ذات معنى أصلاً؟

الإجابة النموذجية

ملاحظة دقيقة: 2NF لا تكون لها مفعول إلا في الجداول ذات المفتاح المركب (composite key)، لأن الاعتماد الجزئي (partial dependency) لا يمكن أن يحدث إلا حين يكون المفتاح من أكثر من عمود. الجداول التي مفتاحها id من نوع BIGSERIAL بسيط (مثل patients, doctors) تحقق 2NF تلقائياً بمجرد تحقيق 1NF.

مواضع المفاتيح المركبة في مشروعِي هي جداول الوصل: role_permissions مفتاحها (role_id, permission_id)، و user_roles مفتاحها (user_id, role_id). هنا تظهر 2NF: الأعمدة الإضافية يجب أن تعتمد على كامل المفتاح. مثال: في user_roles عمود assigned_at يعتمد على الزوج (user_id, role_id) كاملاً — أي لحظة إسناد هذا الدور لهذا المستخدم — لا على user_id وحده ولا على role_id وحده، فهو متوافق مع 2NF.

لو كنت قد وضعت في user_roles عموداً مثل role_description لكان ذلك خرقاً لـ 2NF لأن الوصف يعتمد على جزء من المفتاح (role_id) فقط، والحل نقله إلى جدول roles وهو ما فعلته فعلاً. إذن 2NF عندي محققة بفصل بيانات الدور في roles وبيانات الصلاحيات في permissions وإبقاء جداول الوصل خالية من أي عمود يعتمد على جزء من المفتاح.

اكتب استعلام JOIN يجلب مواعيد مريض معيّن مع اسم الطبيب. وشرح أنواع الـ JOIN ولماذا اخترت نوعك.

الإجابة النموذجية

في مخططي doctor_id في appointments يشير إلى doctors(id)، وبيانات الاسم الشخصية في جدول staff (لأن doctors امتداد 1:1 لـ staff)، لذا نحتاج سلسلة staff → doctors → appointments JOIN: مع patients لاسم المريض. الاستعلام:

```
SELECT a.id AS appointment_id
```

```
,a.scheduled_at
```

```
,a.status
```

```
,p.first_name || ' ' || p.last_name AS patient_name
```

```
,s.first_name || ' ' || s.last_name AS doctor_name
```

```
d.specialty
```

```
FROM appointments a
```

```
JOIN patients p ON p.id = a.patient_id
```

```
JOIN doctors d ON d.id = a.doctor_id
```

```
JOIN staff s ON s.id = d.staff_id
```

```
WHERE a.patient_id = 1
```

```
;ORDER BY a.scheduled_at DESC
```

استعملت INNER JOIN لأن كل موعد له مريض وطبيب إلزاميان (الأعمدة NOT NULL مع FK)، فلا توجد صفوف ناقصة الطرف.

أنواع الـ JOIN باختصار:

- INNER JOIN: يُرجع فقط الصفوف المتطابقة في الطرفين.

- LEFT (OUTER) JOIN: كل صفوف الجدول الأيسر + المتطابق من الأيمن، و NULL إن لم يوجد تطابق. مفيد عندي لو أردت إدراج department: ...LEFT JOIN departments dep ON dep.id = a.department_id لأن department_id قد يكون NULL (FK).
ON DELETE SET NULL

- RIGHT JOIN: عكس LEFT.

- FULL OUTER JOIN: كل الصفوف من الطرفين.

القاعدة: أستخدم INNER عندما تكون العلاقة إلزامية، و LEFT عندما يكون الطرف اختياريًا (nullable FK) كي لا أفقد صفوفًا.

ما الفرق بين الـ VIEW والـ TABLE؟ وأين استخدمت الـ views في مشروعك ولماذا؟

الإجابة النموذجية

الـ TABLE كائن يخزن البيانات فعلياً على القرص. الـ VIEW استعلام مُسمّى محفوظ (stored named query) لا يخزن بيانات بنفسه (في الحالة العادية)، بل يُنفَّذ استعلامه عند كل قراءة ويُرجع نتيجة كأنها جدول افتراضي.

الفروق العملية:

- 1) التخزين: الجدول يحجز مساحة ويحوي صفوفاً؛ الـ view لا يحجز بيانات (إلا materialized view الذي يخزن لقطة ويحتاج (REFRESH).
 - 2) التحديث: الجدول قابل للكتابة دائماً؛ الـ view قابل للتحديث فقط بشروط (simple updatable view)، وغالباً يُستخدم للقراءة.
 - 3) الغرض: الـ view يبسط الاستعلامات المعقدة، ويُعيد استخدام منطق JOIN، والأهم في مشروعك: إخفاء/تقنيع الأعمدة الحساسة (data masking) وأمن العرض.
- استخدامي في المشروع (security.data_masking): أنشأت views لتقييد ما تراه الأدوار الدنيا بدل منحها وصولاً مباشراً للجدول. مثل v_patients_reception لموظف الاستقبال (يظهر الاسم MRN وهاتفاً مقلّعاً عبر mask_phone، ويُخفي address و national_id و البيانات السريرية)، و v_medical_records_nurse للممرّض. ثم REVOKE SELECT على الجدول الأصلي و GRANT SELECT على الـ view فقط، فيستحيل على الدور الأدنى الوصول للحقل الحساس مباشرة.
- الفائدة الأمنية: الـ view هنا طبقة عرض آمنة (least privilege)؛ والـ RLS طبقة دفاع ثانية على مستوى الصف، فيتكاملان (defense in depth).

ما فائدة الـ INDEX، ومتى يكون مضراً؟ اشرح بأمثلة من فهارسك مثل الفهرس المركب والفهرس الجزئي.

الإجابة النموذجية

الـ INDEX بنية مساعدة (غالباً B-tree) تسرع البحث والفرز بتجنب المسح الكامل للجدول (full table scan)، فبدل قراءة كل الصفوف تصل قاعدة البيانات للصف مباشرة في زمن لوغاريتمي تقريباً.

أين يفيد في مشروعي:

- فهارس المفاتيح الأجنبية (FK) المستخدمة في JOIN، قاعدة ذهنية: `idx_appointments_doctor_id`، `idx_medical_records_patient_id`، `idx_prescription_items_prescription` كلها تسرع الـ JOINات وتعبيرات EXISTS داخل سياسات RLS.

- فهرس مركب (composite): `appointments(doctor_id, scheduled_at)`، `idx_appointments_doctor_date` لخدمة استعلام 'جدول مواعيد طبيب مرتبة بالوقت' بفهرس واحد. هنا يهم ترتيب الأعمدة (leftmost prefix rule): يفيد البحث بـ `doctor_id` وحده أو `doctor_id+scheduled_at`، لكنه لا يفيد البحث بـ `scheduled_at` وحده.

- فهرس جزئي (partial): `appointments(scheduled_at) WHERE status='scheduled'`، `idx_appointments_upcoming`، و `idx_invoices_patient_status` ... `WHERE status <> 'paid'`. الفهرس الجزئي يفهرس الصفوف المهمة فقط، فيكون أصغر وأسرع وأقل كلفة صيانة، وممتاز عندما نستعلم دائماً عن مجموعة فرعية (المواعيد القادمة، الفواتير غير المسددة).
- يلاحظ أن UNIQUE و PRIMARY KEY ينشئان فهارسهما تلقائياً، لذا لم أكررها (`username, MRN, license_number`...).

متى يضّر الـ INDEX:

(1) على عمليات الكتابة (INSERT/UPDATE/DELETE): كل فهرس يجب تحديثه مع كل كتابة، فكثرة الفهارس تبطئ الكتابة. في جدول مثل `audit_logs` كثير الإدراج، أبقيت الفهارس على ما يستعلم فعلاً فقط.

(2) المساحة: الفهارس تستهلك تخزيناً إضافياً.

(3) على الأعمدة قليلة التمايز (low cardinality) مثل `gender ('M'/'F')` أو `status` بقيم قليلة، الفهرس العادي قد لا يُستخدم لأن المُحسّن (planner) يفضل المسح الكامل — وهنا يكون الفهرس الجزئي أو المركب أنسب.

(4) فهارس لا تُستخدم أبداً = كلفة بلا عائد.

القاعدة: أفهرس أعمدة الـ WHERE/JOIN/ORDER BY كثيرة الاستخدام، وأتحقق فعلياً بـ EXPLAIN / EXPLAIN ANALYZE أن المُحسّن يستعمل الفهرس ولا يتجاهله.

ما هي خصائص ACID والمعاملات (Transactions)، وأين تظهر أهميتها في نظام المستشفى لديك؟
اربطها بطريقة ضبط سياق RLS.

الإجابة النموذجية

ACID أربع ضمانات للمعاملة (transaction):

- Atomicity (الذرية): المعاملة كُتِل لا يتجزأ — إقفا تُنفَّذ كل عملياتها أو لا شيء (COMMIT أو ROLLBACK). مثالي: عند تسجيل دفعة (payment) قد أُحدِّث invoices.status وأُدرج صفّاً في payments؛ لو فشلت إحدى الخطوتين يجب التراجع عن الأخرى كي لا تبقى فاتورة 'paid' بلا دفعة.

- Consistency (الاتساق): المعاملة تنقل القاعدة من حالة صحيحة إلى حالة صحيحة، محترمةً كل القيود (FK, CHECK, UNIQUE). مثلاً CHECK على `amount > 0` و FK من `payments.invoice_id` لا يُسمح بخرقها داخل المعاملة.

- Isolation (العزل): المعاملات المتزامنة لا يرى بعضها نتائج بعض غير المكتملة. مهم عند حجز موعد: قيد UNIQUE(doctor_id, scheduled_at) مع العزل يمنع حجزين متزامنين لنفس الطبيب في نفس اللحظة (race condition).

- Durability (الديمومة): بعد COMMIT تبقى البيانات محفوظة حتى لو انقطع التيار (عبر WAL في PostgreSQL).

الربط بسياق RLS في مشروع (نقطة مهمة للمناقشة): تطبيقي يلفّ كل عملية في معاملة تبدأ بـ:

```
;BEGIN
```

```
;SET LOCAL app.current_user_id = $1
```

```
;SET LOCAL app.current_role = $2
```

```
-- الاستعلام الفعلي
```

```
;COMMIT
```

استخدام SET LOCAL يجعل قيمة السياق محصورة بهذه المعاملة فقط ويُسمح عند نهايتها — هذا حاسم لأنني أستخدم اتصالاً واحداً مشتركاً عبر connection pool: لو استخدمت SET عادي لتسربت هوية مستخدم إلى طلب مستخدم آخر يعيد استخدام نفس الاتصال (ثغرة خطيرة). فحدود المعاملة هنا تحمي عزل هوية المستخدم وتغذّي دوال RLS مثل `app_current_doctor_id()` و `app_has_role()` ودالة التدقيق `fn_audit()` التي تقرأ نفس المتغيرات لتسجيل من نفَّذ العملية. إذن المعاملة عندي ليست فقط لسلامة البيانات بل هي وعاء أمني يربط كل عملية بهوية فاعلها.

ما هو RBAC (Role-Based Access Control) وكيف طَبَّقته في قاعدة بيانات المستشفى؟

الإجابة النموذجية

RBAC أي «التحكُّم في الوصول المبني على الأدوار» هو نموذج أمني لا تمنح فيه الصلاحية (permission) مباشرةً للمستخدم، بل نمنحها لـ «دور» (role)، ثم نُسند الدور للمستخدم. فيُصبح المسار: $user \leftarrow role \leftarrow permission$.

في مشروعي طَبَّقته عبر أربعة جداول:

(1) roles: الأدوار السبعة (admin, doctor, nurse, receptionist, lab_technician, pharmacist, patient).

(2) permissions: صلاحيات دقيقة بصيغة resource:action مثل medical_records:read و prescriptions:dispense.

(3) role_permissions: جدول وصل N:M يربط كل دور بصلاحياته.

(4) user_roles: جدول وصل N:M يربط المستخدم بأدواره.

الفائدة العملية: إذا أردت تغيير ما يستطيع الطبيب فعله، أَعُدِّل صفوفاً في role_permissions (بيانات data)، دون تعديل كود التطبيق (code). وهذا ما يجعل النظام قابلاً للتوسُّع: إضافة دور جديد = إدراج صفوف، لا برمجة جديدة.

ما الفرق بين role و permission؟ ولماذا لم تربط الصلاحيات بالمستخدم مباشرة؟

الإجابة النموذجية

permission = قدرة ذرية واحدة على فعل محدد، مثل appointments:create (حجز موعد). أما role فهو حزمة (مجموعة) من الـ permissions تمثل وظيفة في النظام، مثل دور doctor الذي يجمع medical_records:create و diagnoses:create و prescriptions:create وغيرها.

لم أربط الصلاحيات بالمستخدم مباشرة لثلاثة أسباب:

(1) تجنَّب التكرار: لدينا عشرات الأطباء؛ لو ربطنا الصلاحيات بكل مستخدم لكرَّرنا نفس القائمة عشرات المرات. مع الـ role نُعرِّفها مرَّة واحدة.

(2) سهولة الصيانة: تعديل صلاحيات الأطباء جميعاً يتم بتعديل دور doctor فقط، فينعكس فوراً على الجميع.

(3) قابلية التدقيق (auditability): يكفي النظر إلى role_permissions لمعرفة قدرات كل دور بوضوح.

بعبارة قاعدة بيانات: العلاقة بين users و permissions هي many-to-many، والـ role هو الكيان الوسيط (intermediate entity) الذي يطع هذه العلاقة ويمنع التكرار (3NF).

لماذا لا تكتفي بفحص الدور في طبقة التطبيق (application layer) وتضيف Row-Level Security داخل قاعدة البيانات؟ أليس ذلك تكراراً؟

الإجابة النموذجية

ليس تكراراً بل مبدأ «الدفاع المتعدد الطبقات» (defense in depth). الفحص في التطبيق وحده غير كافٍ لسببين:

1) فحص التطبيق خشن (coarse-grained): الـ middleware يتحقق هل يملك المستخدم صلاحية `medical_records:read` أصلاً، لكنه لا يدرس «أي صفوف» يراها. فالطبيب يملك الصلاحية، لكن يجب أن يرى سجلات مرضاه هو فقط لا كل المرضى. هذا التقييد على مستوى الصف (row) لا يحسمه الـ permission code وحده.

2) قاعدة البيانات قد تستعلم من مسارات أخرى: سكرتير إداري، أداة تقارير، أو ثغرة SQL injection تتجاوز طبقة التطبيق بالكامل. لو كان الحارس الوحيد في التطبيق، فأني مسار يلتف عليه يكشف كل البيانات.

لذلك جعلت التطبيق يفحص الـ permission (هل يحق له المحاولة أصلاً؟) وجعلت Row-Level Security داخل PostgreSQL يفرض «أي صفوف» يصل إليها فعلاً. لو سقطت طبقة، تبقى الأخرى تحمي البيانات. وقد فُعلت RLS على 7 جداول حساسة (patients, appointments, medical_records, diagnoses, prescriptions, lab_orders, invoices).

ما هو Row-Level Security؟ وكيف يمنع مريضاً من رؤية سجل مريض آخر؟ اشرح بمثال سياسة فعلية من مشروعك.

الإجابة النموذجية

Row-Level Security (RLS) هو ميزة في PostgreSQL تُقيد «أي صفوف» (rows) يراها أو يعدّلها المستخدم داخل الجدول نفسه، عبر سياسة (POLICY) تحتوي شرطاً منطقياً يطبّق تلقائياً على كل استعلام SELECT/INSERT/UPDATE/DELETE. يُفعّل :-

```
;ALTER TABLE patients ENABLE ROW LEVEL SECURITY
```

مثال فعلي من ملفي rbac_rls.sql_02 على جدول patients:

```
) CREATE POLICY patients_select ON patients FOR SELECT USING
```

```
app_has_role('admin') OR app_has_role('receptionist')
```

```
(OR id = app_current_patient_id
```

```
OR EXISTS (SELECT 1 FROM medical_records mr WHERE mr.patient_id = patients.id AND mr.doctor_id =
```

```
app_current_doctor_id()))
```

```
OR EXISTS (SELECT 1 FROM appointments a WHERE a.patient_id = patients.id AND a.doctor_id =
```

```
app_current_doctor_id()))
```

```
;
```

كيف يمنع مريضاً من رؤية غيره؟ السطر id = app_current_patient_id هو المفتاح. الدالة app_current_patient_id() تُرجع patients.id المرتبط بالمستخدم الحالي عبر users.patient_id. فحين يكتب المريض «فاطمة» SELECT * FROM patients، يُضيف PostgreSQL شرط USING تلقائياً، فلا يطابق إلا صفّها هي. ولو حاولت id = 5 SELECT * FROM patients WHERE id = 5 (مريض آخر)، يفشل شرط الـ policy على ذلك الصف فيُجب — تُرجع نتيجة فارغة لا خطأ، وكأن الصف غير موجود. المهم أن المريض لا يحمل دور admin/receptionist ولا doctor، فتسقط كل الفروع الأخرى من الشرط ويبقى صفّه وحده.

كيف تعرف قاعدة البيانات هوية المستخدم الحالي لتطبق سياسة RLS، مع أنك تستخدم اتصالاً واحداً (single connection) لكل التطبيق؟

الإجابة النموذجية

هذه نقطة جوهرية: PostgreSQL لا يعرف «مستخدم التطبيق» لأن كل الطلبات تأتي عبر اتصال واحد بدور قاعدة بيانات واحد هو app_role. الحل هو تمرير سياق المستخدم في كل معاملة (transaction) عبر متغيرات جلسة GUC:

```

;BEGIN
;SET LOCAL app.current_user_id = '12
;SET LOCAL app.current_role = 'doctor
-- الاستعلام الفعلي
;COMMIT

```

استخدمت SET LOCAL تحديداً لأنه يقيّد القيمة بحدود المعاملة الحالية فقط، فلا تتسرب إلى طلب مستخدم آخر يعيد استخدام نفس الاتصال من الـ connection pool.

ثم تقرأ هذه القيم حوالت مساعدة كتبتها في 02_rbac_rls:

- app_current_user_id () تقرأ (app.current_user_id, true) وتحوّلها إلى BIGINT.

- app_has_role(text) هل الدور موجود في القائمة المفصلة بفواصل.

- app_current_doctor_id () تُنفذ JOIN عبر doctors → staff → users لترجع doctor_id.

- app_current_patient_id () تُرجع users.patient_id.

وتستدعي سياسات RLS هذه الدوال. المعامل true في app_current_patient_id يمنع رفع استثناء إن لم يُضبط المتغير، فيرجع NULL بأمان.

كيف يحمل المستخدم الواحد أكثر من دور (multiple roles)؟ وكيف تتعامل سياسات RLS مع مستخدم يجمع دورين، مثلاً طبيب هو أيضاً رئيس قسم بصلاحيات إضافية؟

الإجابة النموذجية

يحمل المستخدم عدّة أدوار عبر جدول الوصل `user_roles` الذي مفتاحه الأساسي مركّب `(user_id, role_id)`، فيقبل عدّة صفوف لنفس `user_id` بأدوار مختلفة — هذه علاقة many-to-many بين `roles` و `users`. في بيانات الـ `seed` عندي كل مستخدم له دور واحد، لكن البنية تسمح بإضافة صفّ ثانٍ لمنحه دوراً إضافياً دون أي تغيير في المخطط. عند تسجيل الدخول، يجمع التطبيق كل أدوار المستخدم من `user_roles` وكل صلاحياتها من `role_permissions`، ويُمزجها معاً. الصلاحيات تتّجّع بالاتحاد (union): المستخدم يحصل على مجموع صلاحيات كل أدواره.

في RLS تعاملت مع تعدّد الأدوار بأمرين:

(1) أمّر الأدوار كقائمة مفصولة بفواصل: `SET LOCAL app.current_role = 'doctor,nurse'`;

(2) دالة `app_has_role` تستخدم `string_to_array` لتقسيم القائمة ثم `p_role = ANY (...)`، فتطابق العنصر كاملاً وتتجنّب التطابق الجزئي الخاطئ (مثلاً ألا يطابق 'admin' دوراً اسمه 'admin_assistant').

وبما أن شروط الـ `policy` مبنية بـ `OR`، فإن المستخدم ذا الدورين يستفيد من أوسع فرع ينطبق عليه: فطبيب-ممرّض يقرأ `medical_records` سواء كصاحب السجّل (`doctor_id = app_current_doctor_id`) أو كممرّض (`app_has_role('nurse')`). أي إن الصلاحيات تتراكم لا تتعارض. ولو احتجّت صلاحيات إضافية لرئيس القسم، أنشئ دوراً جديداً وأُسَيده له كصفّ ثانٍ في `user_roles` — فيجمع المستخدم الفرعين.

ما أكبر خطأ قد يُبطل كل سياسات RLS لديك صامتاً (silently)؟ وكيف حُصّنت مشروعك ضده؟ وما حدود RLS التي اضطررتك لتكميلها بآليات أخرى؟

الإجابة النموذجية

أخطر خطأ هو أن يتصل التطبيق بدور قاعدة بيانات يملك صلاحية SUPERUSER أو سمة BYPASSRLS، لأن هذين يتجاوزان كل سياسات RLS بصمت دون أي رسالة خطأ — فتظن النظام محمياً وهو مكشوف. خطأ ثانٍ مرتبط: مالك الجدول (table owner) لا تُطبّق عليه RLS افتراضياً.

حُصّنت المشروع بثلاثة إجراءات في 02_rbac.sql:

(1) أنشأت دور التطبيق صراحةً بسمات أقل امتياز: `CREATE ROLE app_role LOGIN NOSUPERUSER NOBYPASSRLS NOCREATEDB NOCREATEROLE`.

(2) استخدمت `FORCE ROW LEVEL SECURITY` على كل جدول حسّاس، لا `ENABLE` فقط، حتى تُفرض السياسات حتى على مالك الجدول.

(3) طبّقت مبدأ أقل امتياز (least privilege): منحت `app_role` صلاحيات DML فقط، وسحب `UPDATE/DELETE` عن `audit_logs` (append-only) وعن جداول `(roles, permissions, role_permissions)` لمنع تصعيد الامتياز.

أما حدود RLS التي اضطررتي لتكميلها:

- RLS تعمل على مستوى الصف (row) لا العمود (column). فإخفاء حقل حسّاس مثل `national_id` عن موظف الاستقبال لا يحلّه RLS، بل عالجته بـ `VIEWS` تُقنّع الأعمدة (data masking) أو بـ `GRANT column-level`.

- لا توجد `triggers` على `SELECT` في PostgreSQL، فتدقيق «من قرأ سجلّاً» (read audit) لا يلتقطه `trigger`؛ سجّله من طبقة التطبيق في `audit_logs` بحدث `VIEW`.

- قيود مثل «الممرّض ضمن قسمه فقط» أوضح وأكفأ فرضها في طبقة الخدمة. فالقاعدة العملية: RLS طبقة دفاع ثانية - قوية لتقييد الصفوف، لكنها تُكفّل بالتطبيق والـ `views` والـ `triggers` لتغطية العمود والقراءة والقيود المعقّدة.

لماذا تخزن في عمود password_hash قيمة مجزأة (hash) بدل كلمة المرور نفسها؟ وما الفرق بين التجزئة (hashing) والتشفير (encryption)؟

الإجابة النموذجية

أخزن hash لأن كلمة المرور لو خُزنت كنص صريح (plaintext) فإن أي تسريب لقاعدة البيانات أو وصول للأدمن يكشف كلمات مرور كل المستخدمين فوراً، وكثير منهم يعيد استخدام نفس الكلمة في حسابات أخرى. الـ hash دالة أحادية الاتجاه (one-way)؛ يسهل حساب hash من الكلمة، وبستحيل عملياً عكسه لاستخراج الكلمة. عند الدخول لا أفك الـ hash، بل أجزئ الكلمة المُدخلة وأقارن النتيجة.

الفرق الجوهرى:

- التجزئة (hashing): أحادية الاتجاه، لا تُفك أبداً — تُستعمل لكلمات المرور. في مشروعى العمود password_hash.users.
VARCHAR(255) NOT NULL ويملأ بـ bcrypt.

- التشفير (encryption): ثنائي الاتجاه، يُفك بمفتاح — يُستعمل لبيانات حساسة أحتاج قراءتها لاحقاً مثل national_id (عبر pgcrypto).

خطأ شائع يسأل عنه الدكتور: لا يصح "تشفير" كلمة المرور، لأن التشفير قابل للفك؛ الصحيح تجزئتها. هذا التمييز نقطة تقييم مهمة في مشروعى.

ما خوارزمية التجزئة التي استعملتها؟ وما دور الـ salt ومعامل الكلفة (cost factor)؟ ولماذا جعلت العمود VARCHAR(255)؟

الإجابة النموذجية

استعملت bcrypt. في ملف البذور seed.sql_05 ولدت الـ hash مباشرة في قاعدة البيانات عبر امتداد pgcrypto بالأمر:
crypt('Passw0rd!', gen_salt('bf')) — حيث 'bf' تعني Blowfish أي bcrypt. في طبقة Node/Express يقابلها
bcrypt.hash(password, 12) للإشياء و bcrypt.compare(input, hash) للتحقق.

الـ salt: قيمة عشوائية فريدة تُضاف لكل كلمة مرور قبل التجزئة. فائدته أن مستخدمين بنفس الكلمة يحصلان على hash مختلف، وهذا يُبطل هجمات جداول قوس قزح (rainbow tables) والجداول المُحسوبة مسبقاً. في bcrypt الـ salt يُولّد تلقائياً (gen_salt) ويخزن داخل ناتج الـ hash نفسه فلا أحتاج عموداً منفصلاً له.

معامل الكلفة (cost factor، استعملت 12): يحدد عدد جولات التكرار (2^{12}). كلما زاد صار حساب الـ hash أبطأ عمداً، فتصبح هجمات التخمين بالقوة الغاشمة (brute-force) مكلفة جداً للمهاجم بينما تبقى مقبولة لتسجيل دخول واحد. هذه ميزة bcrypt: بطء مُتعمّد قابل للضبط (adaptive).

VARCHAR(255): ناتج bcrypt 60 حرفاً، لكن وسّعت العمود ليتسع مستقبلاً للترقية إلى argon2id (الموصى به في التصميم، أقوى ضد هجمات GPU) دون تعديل المخطط.

ما ال JWT وكيف يعمل في نظامك؟ وماذا تضع في حمولته (payload)؟ وكيف تتعامل مع تسجيل الخروج وانتهاء الصلاحية؟

الإجابة النموذجية

الـ (JSON Web Token) JWT رمز موقع بثبت هوية المستخدم وصلاحياته دون الحاجة لتخزين جلسة على الخادم (stateless). يتكوّن من ثلاثة أجزاء مفصولة بنقطة: Header.Payload.Signature، كلُّ مُرمّز Base64URL.

آلية العمل في نظامي:

1. تسجيل الدخول POST /api/v1/auth/login: أنُحقّق bcrypt.compare(password, password_hash)؛ وعند النجاح أُصدر access token.
 2. التوقيع: يُوقّع التوكّن بمفتاح سريّ JWT_SECRET (≤ 32 بايت، يُحفظ في متغيّر بيئة لا في الكود) بخوارزمية HS256. التوقيع يمنع التلاعب: أي تعديل على الحمولة يُبطل التوقيع، لأن المهاجم لا يملك المفتاح.
 3. الحمولة (payload): { sub: user_id, staff_id, patient_id, roles: ['doctor'], permissions: ['medical_records:read', 'prescriptions:write'] } . أحقن الأدوار (من user_roles) والصلاحيات (من role_permissions + permissions) داخل التوكّن لأتفادى استعلام قاعدة البيانات في كل طلب.
 4. كل طلب لاحق يرسل التوكّن في ترويسة Authorization: Bearer <token>؛ وميدلوير authenticate ينقّذ jwt.verify ثم يرُكّب req.user.
- الأمان المهم: التوقيع يضمن السلامة (integrity) لا السريّة — الحمولة مقروعة لأي شخص، لذا لا أضع فيها أسراراً (لا password_hash مثلاً).
- انتهاء الصلاحية وتسجيل الخروج: access token قصير العمر (15 دقيقة) لتقليل خطر سرقة، و refresh token طويل (7 أيام) في HttpOnly Secure Cookie لتجديده. لأن JWT بطبيعته لا يُلغى قبل انتهائه، أتعامل مع الخروج عبر قائمة إبطال (blacklist) لا tti في Redis ومسح الـ cookie.

ذكرت أنك شقّرت بعض الحقول بـ pgcrypto. ما هذه الحقول ولماذا اخترتها تحديداً؟ وأين يُحفظ مفتاح التشفير ولماذا لا يُخزّن في قاعدة البيانات؟

الإجابة النموذجية

فُغلت الامتداد بـ CREATE EXTENSION IF NOT EXISTS pgcrypto (موجود في schema.sql_01). اعتمدت تشفيراً انتقائياً للحقول فائقة الحساسية فقط لا تشفيراً شاملاً، لأن التشفير الشامل يعطل البحث والفرز والـ JOIN ويثقل الأداء.

الحقول المختارة: patients.national_id, patients.phone, patients.address, patients.emergency_contact_phone, staff.national_id, إضافةً لمحتوى سريري حسّاس مثل diagnoses.description و medical_records.examination_notes. اخترتها لأنها بيانات تعريف شخصية (PII) أخطرها عند التسرّب هو سرقة الهوية؛ بينما تركت حقولاً كثيرة الاستعلام (vital_signs, chief_complaint) بلا تشفير واكتفيت بحمايتها عبر RBAC و masking.

الآلية: تشفير متماثل بـ AES عبر pgp_sym_encrypt(value, key) عند الإدراج، و pgp_sym_decrypt(col, key) عند القراءة، والحقل يُخزّن بـ BYTEA. للبحث الدقيق على حقل مشفّر (مثلاً بحث الأدمن بـ national_id) أضفت blind index: عمود = encode(hmac(value, key, 'sha256'),'hex') مع فهرس عليه، يسمح بالمطابقة دون فكّ التشفير ودون كشف القيمة.

مفتاح التشفير: يُحفظ في متغيّر بيئة ENCRYPTION_KEY في طبقة التطبيق، ويُحقّن في كل معاملة عبر SET LOCAL app.enc_key = \$1 قبل أي عملية تشفير/فكّ. السبب أنه لو خُزّن المفتاح داخل قاعدة البيانات لانتفت فائدة التشفير كلياً؛ من يسرق البيانات يسرق المفتاح معها. وضعه في GUC جلسة يمنع ظهوره في النسخ الاحتياطية وفي pg_stat. القيد الأكاديمي الذي أذكره: في مشروع طالب مفتاح واحد بلا تدوير يكفي؛ إنتاجياً يلزم نظام إدارة مفاتيح (KMS) مع تدوير دوري.

كيف يعمل التدقيق (audit) في نظامك؟ ولماذا اخترت AFTER trigger؟ وكيف تعرف قاعدة البيانات هوية مستخدم التطبيق أصلاً؟ وكيف ضمنت أن السجل append-only؟

الإجابة النموذجية

جدول audit_logs يلتقط من فعل ماذا على أي سجل ومتى: user_id, action, entity_type, entity_id, old_values JSONB, new_values (اسم الجدول), ('CREATE'/'UPDATE'/'DELETE'/'VIEW'/'LOGIN'), idx_audit_logs_created و idx_audit_logs_entity وهو مفهرس بـ JSONB, ip_address INET, user_agent, created_at. تقارير 'نشاط مريض' و'آخر العمليات'.

آلية التعبئة: دالة trigger عاقبة واحدة (fn_audit) تُربط بالجدول الحساسة (patients, medical_records, diagnoses, prescriptions, lab_orders, invoices, payments, users, ...). تكتب (to_jsonb(OLD) و (to_jsonb(NEW) حسب نوع العملية TG_OP.

لماذا AFTER trigger وليس BEFORE: لأن AFTER ينفذ بعد نجاح العملية فعلاً، فلا أسجل عمليات فشلت أو ارتدت (rolled back). BEFORE قد يسجل ما لم يُكتب أبداً.

كيف تعرف قاعدة البيانات مستخدم التطبيق (المشكلة الكلاسيكية): قاعدة البيانات ترى اتصال app_role واحد لا المستخدم النهائي. الحل: قبل كل عملية كتابة، وداخل نفس المعاملة، ينفذ التطبيق SET LOCAL app.current_user_id = 'id' و SET LOCAL app.client_ip = 'ip'؛ ثم تقرأها الدالة عبر BIGINT عبر current_setting('app.current_user_id', true) — والمعامل true يمنع رفع استثناء إن لم يُضبط المتغير. هذه نفس آلية سياق الجلسة المستعملة في RLS.

تصفية الحساس: عند تدقيق users أستبعد password_hash من to_jsonb كي لا يتسرب الـ hash إلى السجل.

ضمان append-only فعلياً بطيقتين: (1) سحب الصلاحيات على مستوى قاعدة البيانات — في rbac_02.sql نقّدت REVOKE UPDATE, DELETE ON audit_logs FROM app_role, فلا يستطيع التطبيق تعديل/حذف السجل أصلاً؛ (2) trigger رافض (fn_block_audit_change) على BEFORE UPDATE OR DELETE يرفع RAISE EXCEPTION 'audit_logs is append-only'. ملاحظة: SELECT لا يلتقط بـ trigger (PostgreSQL لا يدعم triggers على القراءة)، لذا أحداث 'VIEW' لفتح سجل مريض تُسجل يدوياً من طبقة التطبيق.

ما الـ data masking ولماذا يُطبَّق في نظامك؟ وما الفرق بينه وبين RLS؟ ولماذا اُشترطت أن يكون حساب اتصال التطبيق NOSUPERUSER و NOBYPASSRLS؟

الإجابة النموذجية

Data masking هو إخفاء/تقنيع الأعمدة الحساسة عن الأدوار الدنيا مع السماح ببقية البيانات. طَبَّقته عبر VIEWS تُخفي حقولاً، مع سحب SELECT المباشر عن الجداول الأصلية لتلك الأدوار ومنحه على الـ views فقط (REVOKE SELECT ON patients FROM role_receptionist ثم GRANT على v_patients_reception).

أمثلة من تصميمي:

- موظف الاستقبال: v_patients_reception view يُظهر الاسم و MRN والهاتف مُقَتَّعاً عبر دالة mask_phone (تُظهر آخر 4 أرقام فقط: '1234*****')، ويُرجع NULL لـ national_id و address، ولا يرى medical_records أو diagnoses إطلاقاً.
- الممرِّض: view يرى chief_complaint و vital_signs للرعاية لكن يُخفي عنه المُعَرِّف الوطني والبيانات المالية.
- المريض: view يقيِّد الصفوف بسجله عبر v_patients_reception.

الفرق بين masking و RLS:

- RLS (Row Level Security) يتحكَّم بأي الصفوف يراها المستخدم (التحكم أفقي/على مستوى السطر) — مثلاً 'الطبيب يرى صفوف مرضاه فقط'.

- Masking/views يتحكَّم بأي الأعمدة وكيف تظهر (التحكم رأسي/على مستوى الحقل) — مثلاً 'الاستقبال يرى الهاتف مُقَتَّعاً ولا يرى national_id'.

هما متكاملان: في المشروع استعملت RLS فعلياً على 7 جداول حساسة (CREATE POLICY في 02_rbac.sql)، والـ views/masking كطبقة عرض للأدوار الدنيا. هذا هو الدفاع متعدد الطبقات (defense in depth): التطبيق يفحص permission، و RLS يفلتر الصفوف، والـ view يقنِّع الأعمدة — اختراق طبقة لا يكفي.

لماذا NOSUPERUSER و NOBYPASSRLS لحساب app_role: هذه نقطة حرجية. سياسات RLS لا تُطبَّق على superuser ولا على دور يحمل BYPASSRLS — يتجاوزها بصمت. لو اتصل التطبيق بحساب superuser لانهارت كل حماية الصفوف دون أي خطأ ظاهر، وهو خطأ شائع جداً في المشاريع. لذلك أنشأت app_role بـ NOSUPERUSER, NOBYPASSRLS, NOCREATEDB، وأضفت FORCE ROW LEVEL SECURITY كي تُفرض السياسات حتى على مالك الجدول نفسه.

ما SQL injection وكيف تمنعه في نظامك؟ ولو أفلت استعلام مُحقن، ما الطبقة التي ما زالت تحمي البيانات؟

الإجابة النموذجية

حقن (SQL injection) هجوم يُدخل فيه المهاجم شيفرة SQL ضمن مدخلات المستخدم، فإن دُمجت المدخلات مباشرةً في نص الاستعلام نُفذت كأوامر. مثال: حقن اسم مستخدم بقيمة '1='1' OR في استعلام مبني بالدمج النّصي يحوّل شرط WHERE إلى صحيح دائماً فيتجاوز المصادقة، أو ينقذ DROP TABLE.

المنع الأساسي — الاستعلامات المُعلّمة (parameterized queries / prepared statements): لا أبني SQL بدمج نصوص أبداً، بل أُمزج القيم كمعاملات منفصلة. في pg مع Node:

صحيح: `db.query('SELECT * FROM users WHERE username = $1;', [username])`

خطأ: `db.query("SELECT * FROM users WHERE username = " + username + " ")`

مع \$1 يرسل نص الاستعلام والقيمة منفصلين، فيعامل المحرّك القيمة كبيانات بحتة لا كشيفرة قابلة للتنفيذ — وهذا يُبطل الحقن من جذره. ولاحظ أن نفس النمط (\$1) أستعمله في ضبط سياق `RLS: SET LOCAL app.current_user_id = $1`.

دفاعات مساندة: التحقق من صحة المدخلات (input validation) بمكتبة تحقق، ومبدأ أقل امتياز لـ `app_role` (`NOSUPERUSER`، وسحب صلاحيات خطيرة عبر `REVOKE`) فحتى لو نجح حقن لا يملك الاتصال صلاحية حذف جداول RBAC أو تعديل `audit_logs`، ورسائل خطأ عامة لا تكشف بنية الجداول.

الطبقة الأخيرة إن أفلت الحقن (defense in depth): تبقى RLS حارساً فعلياً. حتى لو تمكّن مهاجم متّصل كـ `app_role` من تنفيذ `SELECT * FROM medical_records`، تطبّق سياسة `medical_records_select` شرط `USING` الذي يقيد الصفوف بـ `doctor_id = app_current_doctor_id` أو `patient_id = app_current_patient_id`، فلا يُعيد إلا الصفوف المسموحة بهوية الجلسة لا كل الجدول. هكذا لا يكفي اختراق طبقة التطبيق وحدها لكشف بيانات كل المرضى.

ما معنى أن الـ API الخاص بك RESTful؟ ولماذا بدأت كل المسارات بـ `/api/v1/`؟

الإجابة النموذجية

REST (Representational State Transfer) نمط لتصميم واجهات الويب يقوم على أربع ركائز طُبِّقَتْها في المشروع: (1) كل شيء مورد (resource) له URL واضح بصيغة جمع اسم، مثل `/patients/` و (2) `appointments` أفعال HTTP تحدد العملية لا الـ GET `/patients/:id`. (3) `getPatient/` فلا أكتب `DELETE=حذف` — `PATCH=تعديل جزئي`، `POST=إنشاء`، `URL: GET=قراءة`، `Stateless` (عديم الحالة): الخادم لا يحفظ جلسة في الذاكرة، بل كل طلب يحمل هويته بنفسه عبر JWT في ترويسة `Authorization`، وهذا يسهّل التوسّع (scalability). (4) رموز حالة HTTP قياسية تصف النتيجة: 200 نجاح، 201 إنشاء، 400 مدخلات خاطئة، 401 غير مصادق، 403 غير مُخوّل، 404 غير موجود، 409 تعارض، 500 خطأ خادم.

أما بدء كل مسار بـ `/api/v1/` فهو إصدار الـ API (versioning) في الـ URI: يتيح لي لاحقاً إصدار `v2` بسلوك مختلف دون كسر العملاء الذين ما زالوا يستعملون `v1` — قاعدة `backward compatibility` (التوافق الخلفي). مثال عملي من المشروع: `POST /api/v1/auth/login` لتسجيل الدخول، و `GET /api/v1/patients/:id` لقراءة ملف مريض.

لماذا فصلت الـ backend إلى طبقات `db / controllers / routes` بدل كتابة كل شيء في ملف واحد؟ ما فائدة هذا الفصل (Separation of Concerns)؟

الإجابة النموذجية

اتبعت مبدأ فصل الاهتمامات (Separation of Concerns): كل طبقة مسؤولة عن شيء واحد فقط:

- طبقة `routes`: تعرّف الـ endpoints وتركّب سلسلة الـ `middleware`. مثال: `router.get('/patients/:id', authenticate, authorize('patients:read'), getPatient)`. دورها التوجيه (routing) وربط الحارس الأمني فقط.
- طبقة `controllers`: تحوي منطق التطبيق (business logic): تستقبل `req`، تتحقق من المدخلات (validation)، تطبّق قيد الملكية (ownership)، وتستدعي طبقة الـ `db` ثم تشكّل الاستجابة. لا تكتب SQL مباشرة.
- طبقة `db`: تتولّى الاتصال بـ PostgreSQL وتنفيذ الاستعلامات عبر `connection pool`، وتلّف كل عملية حساسة في `transaction` مع `SET LOCAL app.current_user_id` (الـ wrapper المركزي).

الفوائد العملية للمناقشة: (1) قابلية الصيانة (maintainability): إذا تغيّر شكل قاعدة البيانات أعدّل طبقة `db` فقط دون لمس المنطق. (2) قابلية الاختبار (testability): أختبر الـ controller بـ `mock` لطبقة `db` دون قاعدة بيانات حقيقية. (3) إعادة الاستخدام (reusability): دالة استعلام واحدة تستعملها عدة (4) `controllers` تجنّب التكرار (DRY): `wrapper` واحد في `db/query.js` يضبط سياق RLS لكل الاستعلامات بدل تكراره. هذه البنية تسمّى أحياناً 3-tier أو MVC مبسّط.

كيف يفرض الـ Express middleware صلاحيات RBAC؟ اشرح الفرق بين authorize و authenticate ولماذا فصلتهما.

الإجابة النموذجية

فرض RBAC يتم عبر middleware-ين متسلسلين، لكلٍ منهما مسؤولية مختلفة (وهذا سبب الفصل):

- 1) authenticate (المصادقة — من أنت؟): يستخلص الرمز من ترويسة <token Authorization: Bearer>، ينفذ `jwt.verify(token, JWT_SECRET)` للتأكد من توقيع وصلاحيات الـ JWT، يتحقق أن الحساب لا يزال نشطاً (`is_active=true`)، ثم يعيئ `{ id, staff_id, patient_id, roles[], permissions[] }` في `req.user`. عند أي إخفاق يرجع `Unauthorized 401`.
- 2) authorize (التحويل — هل يُسمح لك؟): دالة غالياً (higher-order) تأخذ الصلاحيات المطلوبة وترجع middleware: `authorize('patients:read')`. تتحقق أن `req.user.permissions` تحتوي على كل صلاحيات مطلوبة. عند النقص ترجع `Forbidden { error: 'INSUFFICIENT_PERMISSIONS', required: ['patients:read'] }`.

التركيب في المسار: `router.get('/patients/:id', authenticate, authorize('patients:read'), getPatient)` — تُنفذ بالترتيب: أولاً نتأكد من المستخدم، ثم نتأكد أنه مخوّل، ثم يصل للـ `controller`.

لماذا الفصل؟ المصادقة والتحويل مفهومان مختلفان (`Authentication ≠ Authorization`): 401 يعني 'لم أعرف عليك'، و 403 يعني 'عرفتك لكن ممنوع'. كما أن `authenticate` موحد لكل المسارات بينما `authorize` يتغيّر بصلاحيات مختلفة لكل endpoint.

نقطة قوة في التصميم: الصلاحيات الدقيقة (مثل `medical_records:read` resource:action) محفوظة داخل الـ JWT وقت تسجيل الدخول — تُجلب من جداولي `permissions` و `role_permissions` — فلا أستخدم قاعدة البيانات في كل طلب، وأضيف دوراً جديداً ببيانات لا بتعديل الكود.

قاعدة البيانات لا تعرف مستخدم تطبيقك (الاتصال واحد عبر `app_role`). فكيف تجعل سياسات RLS تطبق 'الطبيب يرى مرضاه فقط'؟ اشرح آلية `SET app.current_user_id`.

الإجابة النموذجية

هذه أهم نقطة في التصميم وحلّها بثلاث خطوات:

- (1) المشكلة: كل طلبات التطبيق تصل PostgreSQL عبر اتصال واحد بدور `app_role`، فقاعدة البيانات لا تميّز الطبيب من المريض. لذا أمّر الهوية يدوياً.
 - (2) الحقن داخل المعاملة (transaction): قبل كل استعلام حسّاس أنقذ — `SET LOCAL app.current_user_id = $1;` `BEGIN;` فقط وتنزل بعد `COMMIT;`، `SET LOCAL app.current_role = $2;` الاستعلام. استعمل `SET LOCAL` مهم لأن قيمته تعيش داخل المعاملة (وهذا خطر أمني حقيقي مع `connection pooling`). القيمتان تأتيان من الـ `JWT` (`user_id` والأدوار) عبر `wrapper` مركزي في طبقة `db`.
 - (3) القراءة داخل RLS: عرّفت في `rbac_rls.sql_02` دوال سياق تقرأ هذه المتغيرات: `app_current_user_id` (ترجع `BIGINT` `current_setting('app.current_user_id', true)` (المعامل `true` يمنع رفع استثناء إن لم يُضبط، فيرجع `NULL`)، و `app_has_role(p_role)`، والأهم `app_current_doctor_id` (التي تنقذ `JOIN` عبر السلسلة `users→staff→doctors`). ثم تستعملها السياسة، مثلاً على `medical_records: USING ( doctor_id = app_current_doctor_id() OR patient_id = app_current_patient_id() OR app_has_role('nurse') )`.
- نقطة دفاعية ذكية: `app_current_doctor_id` (ترجع `NULL` إن لم يكن المستخدم طبيباً، فتفشل المقارنة `doctor_id = NULL` تلقائياً (تمنع الوصول بدل أن تفتح — `fail-safe`). هذا 'دفاع متعمّد الطبقات' (`defense in depth`): التطبيق يفحص الصلاحية في `authorize`، وقاعدة البيانات تفرض RLS كطبقة ثانية، فلا يكفي اختراق طبقة واحدة لكشف بيانات مريض آخر.

في ملف 02_rbac_qls.sql أنشأت app_role بخيارات NOSUPERUSER و NOBYPASSRLS واستعملت FORCE ROW LEVEL SECURITY. لماذا كل هذه التفاصيل؟ ماذا يحدث لو أهملتها؟

الإجابة النموذجية

هذه التفاصيل هي ما يجعل RLS فعالاً حقاً لا مجرد كود تجميلي:

(1) NOSUPERUSER و NOBYPASSRLS على app_role: في PostgreSQL الـ superuser — وأي دور يحمل صفة BypassRLS — يتجاوز كل سياسات RLS بصمت. لو وصل تطبيقي بحساب superuser فإن كل سياساتي (الطبيب يرى مرضاه فقط...) تُتجاهل تماماً وأي مستخدم يرى كل البيانات. لذا حساب التطبيق يجب أن يكون دوراً عادياً تُطبق عليه السياسات. هذا خطأ شائع جداً في المشاريع.

(2) FORCE ROW LEVEL SECURITY (بجانب ENABLE): يفعّل السياسات على المستخدمين العاديين، لكن مالك الجدول (table owner) يتجاوزها افتراضياً. FORCE يفرضها حتى على المالك، فيُسدّ منفذ تسرّب عبر حساب المالك.

(3) مبدأ أقل امتياز (Least Privilege): أعطيت app_role فقط ما يلزم — LOGIN، و GRANT SELECT/INSERT/UPDATE/DELETE على الجداول، و USAGE على الـ sequences — ومنعت ما لا يلزم: NOCREATEDB، NOCREATEROLE. وعلى audit_logs نُقّذت REVOKE UPDATE، DELETE لجعله append-only (للإلحاق فقط) فلا يُعبّث بالسجل الأمني، وعلى جداول RBAC (roles, permissions, role_permissions) نُقّذت REVOKE INSERT، UPDATE، DELETE. فلا يعدّلها اتصال المستخدم النهائي ويمنع تصعيد الامتياز. الخلاصة للمناقشة: لو أهملت هذه الخيارات لكانت RLS موجودة في الكود لكنها متجاوزة عملياً — وهذا أسوأ من عدم وجودها لأنه يعطي إحساساً زائفاً بالأمان.

كيف تتعامل مع الأخطاء في الـ backend؟ وكيف تترجم خطأ من قاعدة البيانات (مثل تعارض حجز موعد) إلى استجابة API مفهومة؟

الإجابة النموذجية

اعتمدت معالجة أخطاء مركزية وموحدّة بدل تكرار try/catch في كل مكان:

(1) error-handling middleware مركزي: في Express هو middleware له أربع وسائط (err, req, res, next) يُسجّل آخر السلسلة. أي controller يلتقط خطأ يمرّره بـ next(err)، فيصل لهذا الـ middleware الذي يحوّله لاستجابة موحّدة ويسجّله في الـ logs دون كشف تفاصيل داخلية للعميل.

(2) تنسيق استجابة موحّد: النجاح { success: true, data: {...}, meta: {...} }, والخطأ { success: false, error: { code: ..., message: '...', details: [...] } } — فيتعامل العميل مع شكل ثابت دائماً.

(3) خريطة رموز HTTP الصحيحة: 400 مدخلات خاطئة (فشل validation عبر Zod/express-validator)، 401 غير مصادق، 403 غير مخوّل، 404 غير موجود، 409 تعارض، 429 تجاوز معدّل الطلبات، 500 خطأ خادم غير متوقّع.

(4) ترجمة أخطاء قاعدة البيانات: المثال العملي حجز موعد — القيد UNIQUE(doctor_id, scheduled_at) في الـ schema يرفع خطأ من PostgreSQL برمز 23505 (unique_violation). ألتقطه في طبقة db/controller وأترجمه إلى 409 Conflict برسالة واضحة: 'الطبيب لديه موعد آخر في هذا الوقت' بدل تمرير خطأ SQL الخام. مثال آخر: نقص مخزون دواء عند صرف وصفة يرجع 409 كذلك.

مبدأ أمني مهم: لا أفصح عن تفاصيل داخلية في رسائل الخطأ. في فشل الدخول أرجع 'اسم المستخدم أو كلمة المرور غير صحيحة' دون تمييز أيهما خاطئ، لمنع تعداد الحسابات (account enumeration). و 500 لا يكشف stack trace للعميل بل يُسجّل داخلياً فقط.

كيف تحمي البيانات الحساسة مثل كلمة سر قاعدة البيانات و JWT_SECRET ومفتاح التشفير؟ ولماذا لا تكتبها في الكود؟

الإجابة النموذجية

أفضل كل الأسرار عن الكود وأضعها في متغيرات بيئة (environment variables) عبر ملف env. يُحَقَّل بمكتبة dotenv وأقرأها في الكود بـ process.env:

- DATABASE_URL (وفيه كلمة سر app_role) مع sslmode=require لتشفير قناة الاتصال بقاعدة البيانات.

- JWT_SECRET قوي (32 بايت فأكثر) لتوقيع رموز JWT.

- ENCRYPTION_KEY لتشفير حقول PII الحساسة عبر pgcrypto، ويُحَقَّن في الجلسة عبر app.enc_key SET LOCAL فلا يُخزَّن في قاعدة البيانات.

لماذا لا في الكود؟ (1) الكود يُرفع إلى Git؛ كتابة سرّ فيه تعني تسريبه لأي من يصل المستودع، ويبقى في تاريخ الـ commits حتى بعد حذفه (hardcoded secret خطر شائع). (2) فصل الإعدادات عن الكود مبدأ Factor App-12: نفس الكود يعمل بإعدادات مختلفة بين بيئة التطوير والإنتاج بتغيير env فقط. (3) الفصل يسمح بتدوير المفاتيح (key rotation) دون تعديل الكود. إجراءات الحماية العملية في المشروع: (1) أضع env. في gitignore. فلا يُرفع أبداً، وأرفع بدلاً منه env.example بالمفاتيح بقيم فارغة كتوثيق. (2) في الإنتاج أغير القيمة الافتراضية 'change_me_in_production' الواردة لـ app_role في السكريبت. (3) كلمات مرور المستخدمين ليست سرّاً مخزّناً بل تُجرّأ one-way لـ bcrypt (cost=12) أو argon2id، فلا تُفكّ أبداً — وهذا يختلف عن أسرار البيئة القابلة للاستعمال. (4) إضافات تشدّد الأمان: Helmet للترويسات الأمنية، فرض HTTPS، و Refresh Token في HttpOnly + Secure + SameSite=Strict cookie لمنع سرقة XSS.

لماذا اخترت موضوع 'نظام قاعدة بيانات مستشفى آمن مع RBAC؟ وما المشكلة الحقيقية التي يحلّها؟ (Why this project / problem statement)

الإجابة النموذجية

اخترت هذا الموضوع لأنه يجمع بين تحدّي تقني حقيقي ومجال حسّاس فعلاً. بيانات المستشفى (السجل الطبي medical_records، التشخيص diagnoses، الوصفات prescriptions) من أكثر البيانات حساسية، وتسريبها يعني انتهاكاً لخصوصية المريض. المشكلة العملية: في أنظمة كثيرة كل مستخدم يصل لكل شيء، فموظف الاستقبال قد يقرأ تشخيص أي مريض، وهذا خطأ أمني.

المشروع يحلّ ذلك بمبدأ أقل امتياز (least privilege) عبر RBAC: لكل دور (7 أدوار: admin, doctor, nurse, receptionist, lab_technician, pharmacist, patient) صلاحيات دقيقة بصيغة resource:action مثل medical_records:read. ولأن فحص التطبيق وحده غير كافٍ، أضفت طبقة ثانية في قاعدة البيانات نفسها هي Row Level Security (RLS)، فيصبح لدينا دفاع متعدد الطبقات (defense in depth): حتى لو اخترق أحدهم طبقة التطبيق، تمنعه قاعدة البيانات من رؤية صفوف ليست له. باختصار: الموضوع واقعي، يطبّق تطبيق 3NF سليماً (22 جدولاً)، ويظهر مفاهيم أمان متقدّمة (RBAC + RLS + hashing + audit) في سياق له معنى.

كيف تختبر عملياً أنّ نظام الـ RBAC والـ RLS يعمل فعلاً؟ صف خطوات اختبار ملموسة. (How do you test RBAC works)

الإجابة النموذجية

- أختبر على مستويين: إيجابي (المسموح ينجح) وسليبي (الممنوع يُرفض)، والاختبار السليبي هو الأهم في الأمان.
- الخطوات العملية باستخدام البيانات التجريبية (seed.sql_05):
- 1) أفتح جلسة واضبط سياق المستخدم كما يفعل التطبيق: `SET app.current_user_id = '<id>'; SET dr.ahmed>;` ثم `SELECT * FROM medical_records`؛ `app.current_role = 'doctor'`؛
 - 2) الاختبار الإيجابي: الطبيب أحمد يجب أن يرى سجلات مرضاه فقط (المرتبطين به عبر doctor_id) ولا يرى سجلات مرضى الطبيب الآخر.
 - 3) الاختبار السليبي الحاسم: أضبط الدور إلى receptionist ثم `SELECT * FROM medical_records`؛ — يجب أن يرجع صفراً من الصفوف (موظف الاستقبال لا سياسة قراءة له على السجل السريري).
 - 4) اختبار المريض: `app.current_user_id` للمريض، يجب أن يرى سجله هو فقط لا سجل غيره.
 - 5) اختبار حدود الكتابة: المريض يحاول UPDATE موعد بحالة غير cancelled — يجب أن يُرفض بـ WITH CHECK، بينما إلغاؤه (status='cancelled') ينجح.
 - 6) اختبار append-only: محاولة UPDATE أو DELETE على audit_logs يجب أن تُرفض.
- أتمتة ذلك بمجموعة اختبارات (مثلاً Jest + supertest على الـ API، أو سكريبت SQL) تنتقل بين الأدوار وتتحقق من النتائج المتوقّعة لكل حالة. النجاح يعني: كل مسموح يمرّ، وكل ممنوع يُرفض.

ما أصعب تحدّي تقني واجهته في المشروع وكيف حللته؟ (Hardest challenge and how you solved it)

الإجابة النموذجية

أصعب تحدّي كان: 'قاعدة البيانات لا تعرف من هو مستخدم التطبيق'. التطبيق يتصل بـ PostgreSQL عبر اتصال واحد بدور app_role، فكيف تعرف سياسات RLS أنّ الطلب الحالي يخصّ الطبيب أحمد لا غيره؟

الحل الذي اعتمدته هو سياق الجلسة (session context) عبر متغيّرات GUC: قبل كل استعلام، وداخل نفس المعاملة (transaction)، يضبط التطبيق:

```
<SET LOCAL app.current_user_id = 'users.id';
```

```
SET LOCAL app.current_role = 'doctor';
```

ثم كتبت دوال مساعدة تقرأ هذا السياق: app_current_user_id() و app_has_role(text)، والأهم

app_current_doctor_id() التي تنقذ JOIN عبر السلسلة users→staff→doctors لترجع مُعرّف الطبيب.

النقطة الذكيّة في الحل: هذه الدالة تُرجع NULL إذا لم يكن المستخدم طبيباً، فتفشل كل مقارنات doctor_id = app_current_doctor_id بأمان — أي أنّ السلوك الافتراضي عند الغموض هو المنع لا الفتح (fail-safe / fail-closed). واستخدمت SET LOCAL لا SET كي ينتهي السياق تلقائياً بانتهاء المعاملة فلا يتسرّب لطلب آخر في نفس الـ connection pool.

تحدّي ثانٍ مرتبط: ضبطت الدوال بـ STABLE و SECURITY INVOKER وحددت search_path صراحةً لمنع function-hijacking، وجعلت app_role دوراً عادياً بـ NOBYPASSRLS لأن أيّ superuser يتجاوز RLS بصمت.

ما حدود نظامك (limitations)؟ وما الذي يعمل في التصميم فقط ولم يُنفذ بالكامل؟ (System limitations / honest boundaries)

الإجابة النموذجية

أذكر الحدود بصراحة لأنها تُظهر فهماً ناضجاً لا ضعفاً:

- 1) قيود لا يفرضها RLS: RLS يعمل على مستوى الصف (row) لا الحقل (column). فقيود مثل 'الممرّض يعدّل vital_signs فقط لا التشخيص' لا يفرض كاملاً بـ RLS وحده — يحتاج column-level GRANT أو فحصاً في طبقة التطبيق. وكذلك 'الممرّض ضمن قسمه فقط' هو حالياً مبسّط (أي ممرّض يقرأ السجل السريري). هذه قيود موثقة تُفرض منطقياً في التطبيق.
- 2) admin يحمل صلاحيات إدارية واسعة، وفصل الواجبات (SoD) مقترح عبر دور auditor للقراءة فقط لكنه لم يُفعّل بالكامل.
- 3) حقن صلاحيات RBAC داخل JWT يعني أنّ تغيير دور المستخدم لا يسري إلا بعد إعادة إصدار التوكن (حتى انتهاء 15 دقيقة).
- 4) التشفير: التصميم يقترح pgcrypto للحقول الحساسة (مثل national_id) بمفتاح واحد بلا تدوير؛ إنتاجياً يلزم KMS. وال seed الحالي ينقذ تجزئة كلمات المرور (bcrypt) لكن تشفير حقول PII قد يكون مرحلة لاحقة.
- 5) تدقيق القراءة (VIEW): PostgreSQL لا يدعم trigger على SELECT، لذا 'من اطلع على ماذا' يجب تسجيله من طبقة التطبيق يدوياً.
- 6) النظام مُختبر ببيانات تجريبية صغيرة (8 مرضى، 9 موظفين) لا بحمل إنتاجي حقيقي.

لو توقّر لديك وقت أكثر، ما الذي ستُحسّنه أو تضيفه؟ (Future work)

الإجابة النموذجية

أولويات التطوير المستقبلي مرتّبة حسب العائد الأمني والقيمة:

(1) تفعيل audit triggers موحّدة: دالة fn_audit() واحدة بـ SECURITY DEFINER تُربط على كل الجداول الحسّاسة (AFTER INSERT/UPDATE/DELETE) فتسجّل old_values/new_values كـ JSONB تلقائياً حتى لو نسي كود التطبيق ذلك، مع استبعاد password_hash من السجل. هذه أعلى أولوية لأنها تكمل خاصية التتبّع.

(2) تشفير حقول PII الحسّاسة (national_id, phone, address) عبر pgcrypto مع Blind index (HMAC) للبحث الدقيق دون فكّ التشفير.

(3) إدارة جلسات حقيقية: جدول user_sessions لتخزين refresh tokens مجرّأة ودعم الإبطال (logout/revoke)، وقفل الحساب بعد المحاولات الفاشلة (lock_until) بدل التعطيل الدائم.

(4) Data masking عبر views: عرض مقنّع للهاتف/المعرّف الوطني لموظف الاستقبال (mask_phone) مع REVOKE على الجداول الأصلية.

(5) دور auditor فعلي لفصل الواجبات، وترقية التجزئة من bcrypt إلى argon2id.

(6) تحسينات أداء: فهرس GIN trigram للبحث بالاسم، ومراقبة الاستعلامات بـ EXPLAIN ANALYZE.

(7) واجهة أمامية (frontend) وحماية Docker للنشر، ومجموعة اختبارات آلية أوسع للحالات السلبية.

لو كبر النظام إلى مليون مريض وملايين السجلات، ماذا تفعل للحفاظ على الأداء والأمان؟ (Scaling to one million patients)

الإجابة النموذجية

أتعامل مع التوسع على عدة محاور:

- 1) الفهرسة (الأهم فوراً): المخطط يفهرس كل المفاتيح الأجنبية المستخدمة في JOIN (medical_records.patient_id/doctor_id, appointments.doctor_id...) الحرج هنا أن سياسات RLS نفسها تستخدم EXISTS مع هذه المفاتيح، فبدون الفهارس يتحول كل فحص RLS إلى مسح كامل للجدول (sequential scan). لذلك أتحقق بـ EXPLAIN ANALYZE أن السياسات لا تلغي استخدام الفهارس، وأضيف فهارس مركبة وجزئية مثل 'idx_invoices_patient_status WHERE status<>'paid و idx_appointments_doctor_date.
- 2) التقسيم (Partitioning): الجداول الضخمة المتنامية مثل audit_logs و medical_records أقسمها زمنياً (partition by range على created_at/visit_date) بحيث تعمل الاستعلامات على قسم واحد لا الجدول كله، وأرشفة الأقسام القديمة.
- 3) Connection pooling: لأن التطبيق يضبط app.current_user_id لكل معاملة، استخدم PgBouncer بنمط transaction pooling مع SET LOCAL ضروري لتحمل آلاف الاتصالات المتزامنة.
- 4) القراءة المكثفة: read replicas للتقارير والقراءات الثقيلة، وفصلها عن الكتابة.
- 5) Caching: حقن الأدوار والصلاحيات في الـ JWT (موجود) يلغي استعلام DB لفحص الصلاحية في كل طلب؛ وكاش Redis لحالة الحساب.
- 6) الأمان مع التوسع: لا أساوم على RLS و FORCE ROW LEVEL SECURITY مهما كبر الحجم؛ بل أضمن بقاء app_role بـ NOBYPASSRLS، وأراقب أداء دوال السياق (STABLE) كي لا تُنفَّذ مراراً داخل الاستعلام.
- 7) النسخ الاحتياطي والمراقبة: نسخ احتياطي منتظم مشقّر، ومراقبة pg_stat_statements لاكتشاف الاستعلامات البطيئة.

سؤال 'ماذا لو...' مفحّخ: لو نسي المطوّر أن يضبط `app.current_role` قبل الاستعلام، أو لو اتّصل التطبيق بحساب `superuser` بدل `app_role` — ماذا يحدث لأمان النظام؟ (What-if: missing session / superuser connection)

الإجابة النموذجية

هذا سؤال جوهري يكشف فهم حدود التصميم، وأجيب عن الشّقين:

الحالة 1 — نسيان ضبط `app.current_user_id` / `app.current_role`:

هنا يظهر تصميمي الدفاعي (fail-safe). الدوال تستخدم (`'app.current_role', true`) حيث `current_setting` يعنى `true` يمنع رفع استثناء ويُرجع `NULL`/سلسلة فارغة. النتيجة: `app_has_role` (...) تُرجع `FALSE`، و `app_current_doctor_id()/app_current_patient_id` تُرجعان `NULL`، فتفشل كل شروط الـ `USING`. أي أنّ المستخدم بلا سياق لا يرى شيئاً (zero rows) بدلاً من أن يرى كل شيء. هذا هو السلوك الصحيح: الغموض = منع، لا فتح. فالنسيان يكسر الوظيفة لكنه لا يفتح ثغرة تسريب.

الحالة 2 — الاتصال بحساب `superuser` (أو دور بـ `BYPASSRLS`):

هذه كارثة أمنية صامتة. PostgreSQL يتجاوز كل سياسات RLS تلقائياً لأيّ `superuser` أو دور يحمل `BYPASSRLS`، دون أي خطأ ظاهر — فيرى الاتصال كل صفوف كل الجداول. ولهذا في القسم (د) أنشأت `app_role` صراحةً بـ `NOSUPERUSER` و `NOBYPASSRLS`، وفعلت `FORCE ROW LEVEL SECURITY` (لا `ENABLE` فقط) كي تُفرض السياسات حتى على مالك الجدول. وهذا أكثر خطأ شائع يفشل أنظمة RLS في المشاريع: استخدام حساب `admin` للاتصال.

الخلاصة: نسيان السياق يفشل بأمان (fail-closed)، أمّا الاتصال بـ `superuser` فيتجاوز الحماية كلها بصمت — ولذلك حرصت اتصال التطبيق بدور عادي محدود الامتياز، وهذه نقطة دفاع متعمّدة في التصميم لا مصادفة.